MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Single-GPU mixture-of-experts inference can require weight reads from a cold tier when expert residency is limited. MECC combines a placement interface with joint HBM/host retention and weight delivery. We evaluate its current runtime on one A100 with Mixtral-8x7B and a RAID block device reported as rotational. With uniform initial placement and eight WikiText-2 prompts, MECC takes 7.38 s per decode token, versus 13.04 s for unoptimized streaming and 21.75 s for a fixed-CPU cache. A local recency/frequency adapter takes 7.19 s. A matched independent-cache control takes 32.94 s. These single-run measurements support the retention comparison; profile-guided placement and cross-task adaptation remain separate experiments.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models select a subset of experts for each token [1]–[3]. Mixtral and DeepSeekMoE instantiate sparse routing in language models [4], [5]. A single GPU can execute selected experts without full HBM residency, provided each weight is delivered before computation.

Shared servers assign each service a host-memory allowance. When combined HBM and host residency cannot hold the expert pool, requests reach HDD. FlexGen and DeepSpeed Inference already plan heterogeneous-memory execution [6], [7]; flash-aware inference also addresses models exceeding DRAM [8]. We study expert delivery under fixed task caps.

Prior work uses CPU/GPU orchestration [9], memory-constrained planning [10], and SSD-native delivery [11]. MoE-Infinity traces activations for cache replacement and prefetch [12]. ProMoE predicts future expert use [13]. Learned replacement [14] and domain-guided residency [15] further exploit routes. We compare placement, retention, and scheduling under common physical caps.

Three coupled decisions govern expert delivery. **First, capacity has to be distributed across layers.** Uniform slots can leave some layers repeatedly reading from HDD. **Second, HBM eviction has to preserve later reuse.** A fixed CPU pool may discard useful experts, while independent tier caches can retain duplicate copies. **Third, residency has to track request changes.** An initial profile covers startup demand; online observations update that placement as the working set changes.

Our design treats the route profile as an initial residency prior and actual requests as online corrections. MECC assigns these roles across HBM, CPU pinned memory, and HDD. The compiler assigns expert rankings and layer capacities. The runtime promotes requested experts and demotes HBM victims into the host working set. Transfer overlap requires independent work and enforced readiness before computation. Section III presents the framework; Section V defines physical accounting and metrics.

We measure the current runtime under uniform initialization and compare its cache behavior with single-GPU local controls. The profile compiler and multi-GPU resident alternative are not timed in this evaluation.

II. Deployment Model and Challenges

A. Single-GPU Execution with Bounded Host Memory

Consider L MoE layers. Layer ℓ contains N_ℓ experts and selects k_ℓ per token. Expert $e = (\ell, j)$ is an immutable weight object of size s_e. Dense weights, activations, KV cache, and temporary buffers also consume HBM. The task is assigned nominal expert slots in HBM and host memory; mirrors and staging consume additional physical memory.

When selected weights are absent from HBM and host memory, a request reads the backing store before host-to-device transfer. Cold reads can enter both prefill and decode. We distinguish demand-tier counts from physical block-device bytes and record the phase of each request.

B. Initial Coverage and Retention

Uniform per-layer slots may spend capacity on rarely selected experts. A route profile can rank experts within each layer and guide initial placement, but its benefit must be measured against the same online replacement rule. The current main comparison uses uniform quotas and natural expert order; it does not establish a profile-placement gain.

An HBM victim can still be useful to later tokens. Independent HBM and host caches may duplicate experts or fail to preserve a victim in the host working set. Joint promotion and demotion can improve coverage under the same nominal slot counts. The matched independent-cache result in Section VI-C tests this mechanism on one workload.

C. Workload Change

The requested experts can change with the input stream. A compiler profile provides an initial prior, while runtime recency reflects later requests. A separate experiment must use distinct task inputs and report both sides of the switch. No task-switch performance claim is made from the current main runs.

III. Overview

Figure 1 separates offline compilation from online execution. An independent route profile, model metadata, task memory caps, and delivery costs define the compilation inputs. The output specifies layer capacities and initial expert IDs. Mirrors and staging are charged outside unique primary residency; HDD retains all backing weights.

The unchanged router supplies demand IDs. Residency lookup selects a source tier; the runtime promotes requested experts and demotes HBM victims. The delivery path enforces weight readiness before execution. Layer capacities stay fixed during a run, while expert identities and recency change. Planner controls therefore fix online rules; replacement controls fix quotas and initialization (Section VI-C).

IV. Method

A. Compiler: Profile-Guided Placement

The compiler takes model metadata, memory budgets, delivery costs, and an independent route profile. The profile supplies request probability p_e for each expert. For equal-size experts within a layer, a discrete Zipf model summarizes the ranked probabilities:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{H_{N_\ell}(\alpha_\ell)}, \qquad H_n(\alpha) = \sum_{r=1}^{n} r^{-\alpha}. \tag{1}$$

Here $q_{\ell,r}$ is conditional on an expert selection in layer ℓ. Let w_ℓ be that layer's fraction of all selections. The global probability is $p_e = w_\ell q_{\ell,r}$, with $\sum_\ell w_\ell = 1$. For fixed top-k routing, $w_\ell = k_\ell / \sum_j k_j$. Each rank retains its profiled expert ID. Empirical probabilities can replace the fitted Zipf distribution. Neither representation encodes request order.

Let $x_{e,t} \in \{0, 1\}$ indicate the primary tier of expert e. The tiers are HBM, CPU, and HDD, denoted by H, C, and S. Residency budgets $\bar{B}_t$ must exclude buffers and mirrors. The following static objective is a reference model, not a claim that the current planner solves the full problem:

$$\begin{aligned} \min_x \quad & \sum_e p_e \sum_t x_{e,t} T_t(s_e), \\ \text{s.t.} \quad & \sum_t x_{e,t} = 1, \quad \sum_e s_e x_{e,t} \le \bar{B}_t \quad (t \in \{\mathrm{H}, \mathrm{C}\}). \end{aligned} \tag{2}$$

The affine model $T_t(s) = a_t + s/b_t$ represents fixed overhead and effective bandwidth. Calibration uses the target machine. Equation 2 estimates static delivery cost under the profile. Runtime locality, migration, and overlap are evaluated separately.

Proposition 1: Assume equal-size objects, fixed request probabilities, independent tier capacities, and $T_\mathrm{H} \le T_\mathrm{C} \le T_\mathrm{S}$. An optimal static placement fills HBM with the most probable objects, CPU with the next most probable objects, and HDD with the remainder.

Proof sketch. Suppose $p_i > p_j$ and object i occupies the slower tier. Exchanging the two objects changes expected cost by $(p_i - p_j)(T_\mathrm{fast} - T_\mathrm{slow}) \le 0$. Repeated exchanges produce ranked placement. Filling an empty faster-tier slot also cannot increase cost. The result applies to the stated static model.

The multilayer implementation allocates slots by marginal coverage. Under the Zipf model, a slot covering rank r adds

$$\Delta_\ell(r) = w_\ell q_{\ell,r}. \tag{3}$$

This is a coverage score, not the exact latency marginal of Equation 2. Tier-boundary movements can affect both HBM and CPU assignments. The heuristic allocates HBM slots first, then CPU slots from the remaining ranks. At each step it selects the largest next-rank score; ties use layer ID. Expert-ranking ties use expert ID. It stops at the tier's slot cap or exhaustion of experts. Per-layer minima, if enabled, are assigned first and an infeasible minimum allocation must be rejected. A direct scan costs $O(L(S_\mathrm{H} + S_\mathrm{C}))$ after ranking, where S_t is a total slot cap. These rules define the compiler interface. Batched prefill additionally requires distinguishing token selections from distinct expert fetches. The sequential heuristic does not solve the general static objective.

The plan records per-layer capacities, ranked expert IDs, initial tiers, and modeled hit fractions. A missing profile selects natural expert order. The measured runs below use this uniform/natural-order path, so they test the runtime retention and delivery stack rather than the profile compiler. Profile initialization cost and gains require separate matched runs.

B. Runtime: Cross-Tier Retention

Each layer has ordered HBM and CPU residency maps. An HBM hit updates recency. A CPU hit removes the expert from CPU residency and promotes it to HBM as most recently used. An HDD miss reads the weights into staging before H2D transfer. When HBM is full, its least-recently used expert is demoted to CPU. CPU pressure evicts the least-recently used host resident. The HDD backing weights remain available.

Figure 2 starts with HBM $[e_0, e_1]$ and CPU $[e_2, e_3, e_4]$. Requesting e_2 promotes it and demotes e_0. The next request for e_0 hits CPU. This example uses exclusive logical residency. An implementation can keep a CPU mirror to avoid device-to-host (D2H) transfer on eviction. The mirror consumes host memory and is charged separately.

For a serial demand stream within one layer, concatenate the HBM MRU-to-LRU stack with the CPU MRU-to-LRU stack. Exclusive promotion and victim demotion preserve the corresponding LRU stack of

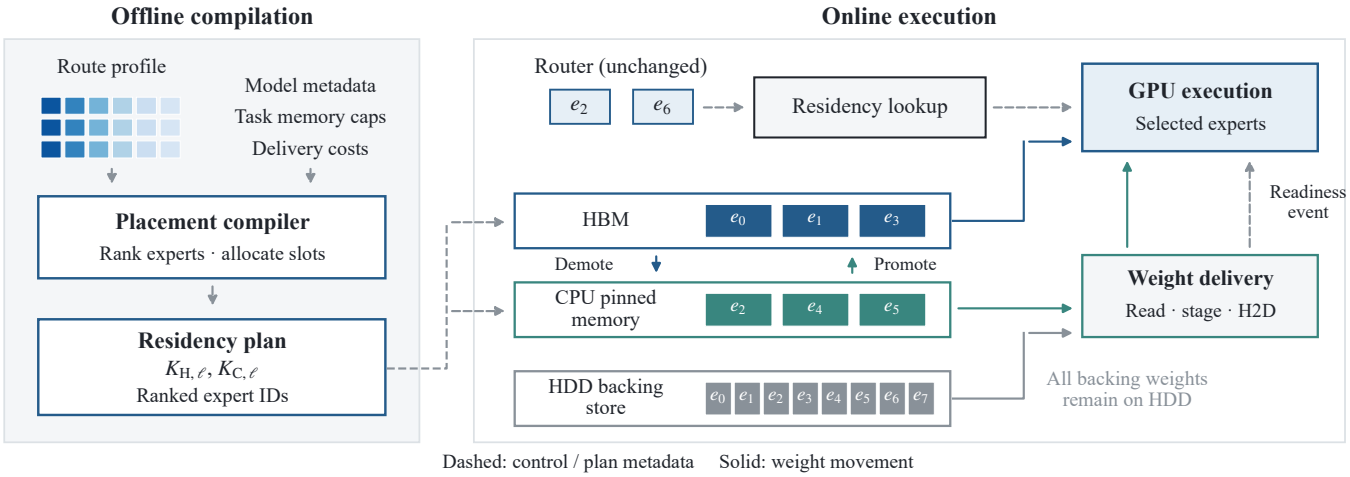


Fig. 1. MECC framework. Offline compilation emits capacities $K_{\mathrm{H},\ell}$ and $K_{\mathrm{C},\ell}$ and ranked expert IDs. Online lookup resolves unchanged router selections against HBM, CPU pinned memory, and HDD. Weight delivery records a readiness event before GPU execution. Dashed arrows carry control/plan metadata; solid arrows move weights. Expert IDs and occupancy are schematic.

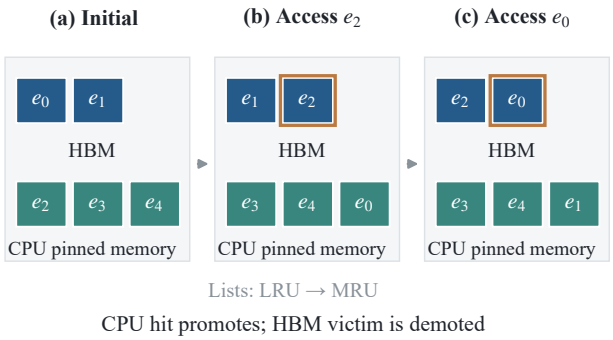


Fig. 2. Cross-tier retention with two HBM slots and three CPU slots. Lists run from LRU to MRU. A CPU hit promotes the requested expert and makes room for the HBM victim. Mirrors and staging consume additional physical space. Schematic.

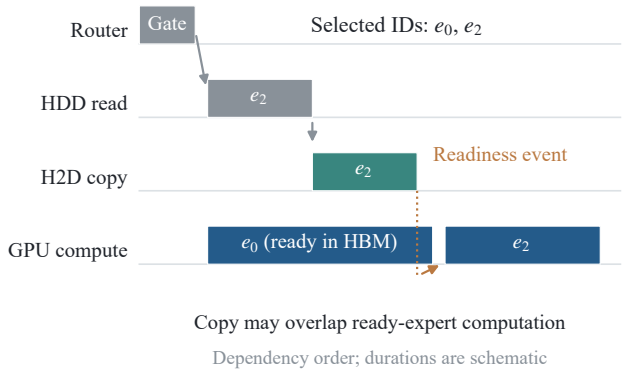


Fig. 3. Delivery dependencies after a gate selects e_0 and e_2. The resident e_0 can execute while e_2 is read and copied. The readiness event protects consumption of e_2. Kernel order is unchanged. Durations are schematic; the figure asserts no measured overlap.

capacity $K_H + K_C$. This reference requires matching initialization, no prefetch or alternative admission, and completed transitions before the next request. It does not imply equivalence to a cache that reallocates capacity across layers, or dominance over static placement.

C. Delivery: Dependency-Aware Overlap

A correct asynchronous implementation requires a readiness event after each pinned-host copy. The computation stream must wait before consuming the weight. The source buffer must remain live until copy completion, and the resident weight until its last consumer finishes. These are required lifetime rules, including across separate loading and dispatch iterators; the execution stream must enforce them with readiness events.

At batch one, routing reveals the selected IDs only after the current layer's gate. Copying one selected expert may overlap computation of another already-ready expert if kernel and aggregation order permit. This provides no knowledge of a later layer's route. In-flight weights must be protected from eviction. Replacement comparisons must disable predictive prefetch; the overlap control must hold admission, residency, and requests fixed. The available overlap and its exposed critical path remain measurements, rather than an assumed gain from a copy stream.

Figure 3 pairs one HBM-resident expert with one HDD-backed expert from the same gate. HDD read precedes H2D and readiness. Copy completion releases the pinned source; final-consumer completion permits GPU eviction. Useful independent computation determines the available overlap.

The design requires unchanged weights, selected experts, kernels, and aggregation order. Under these conditions, changing residency preserves the forward computation. The implementation also needs correct synchronization. The validation protocol compares synchronous and asynchronous paths on identical inputs using weight hashes, route selections, logits, tokens, and perplexity; the same protocol isolates transfer scheduling from changes in numerical execution.

V. Implementation

Model adapters map layer/expert IDs to weight objects, and policies share an end-to-end loader. The harness maps each layer/expert ID to an immutable FP16 weight object. Policies share the same loader and router interface. Memory accounting includes physical mirrors and staging buffers.

Physical accounting deduplicates storage identities, not expert IDs: distinct live copies still consume bytes. Let R_t be resident expert bytes, D_H distinct host mirrors, S_t staging, and O_t other charged allocations. The required constraints are

$$R_C + D_H + S_C + O_C \le B_C, \tag{4}$$

$$R_H + W_{\text{dense}} + W_{\text{KV/act}} + S_H + O_H \le B_{\text{GPU}}. \tag{5}$$

Full mirroring requires $D_H \ge R_H$. Residency must be reduced or a mirror-free path validated. Peak measurements must include initialization and transient copies, not only steady residency.

Counters must separate initialization, warmup, prefill, decode, and quality evaluation. TPOT divides decode time by measured intervals; TTFT includes prefill and the first output. Demand-miss share is the fraction of selected-expert arrivals without ready HBM/CPU weights. In-flight prefetch is logged separately. Physical HDD/H2D/D2H bytes use the same phase boundaries. These counters define the same phase boundaries for every policy.

VI. Evaluation

A. Testbed and Comparison

The model is Mixtral-8x7B-Instruct-v0.1 with FP16 expert weights on one A100 80 GB PCIe GPU. The backing store resides on a RAID logical block device reported as rotational (`ROTA=1`); the physical drives behind the controller have not been inventoried. We call the cold tier HDD as a deployment shorthand. No SSD/NVMe comparison is measured.

Each main policy has one completed end-to-end run on the same model revision with nominal expert allowances of 50 GB HBM and 40 GB host memory, four HBM and three host slots per layer, uniform initialization, and cold reads enabled. The input source is WikiText-2 test: eight prompts truncated to 64 input tokens, with 64 generated tokens each. Perplexity uses a 1024-token WikiText-2 sample. Values are single-run point estimates. The nominal host allowance is a planner parameter; mirrors, staging, and page cache are not demonstrated to fit within a physical 40 GB process cap.

The compared policies are unoptimized streaming (`naive`), fixed-CPU retention (`swapmoe`), MECC, a local recency/frequency adapter, a CPU-compute adapter, and MECC with an alternative I/O path (`ours_io`). The RF and I/O adapters are local implementations, not reproductions of published FlashMoE or SSD-LLaMA. These six runs use one A100 each. There is no measured multi-GPU reference.

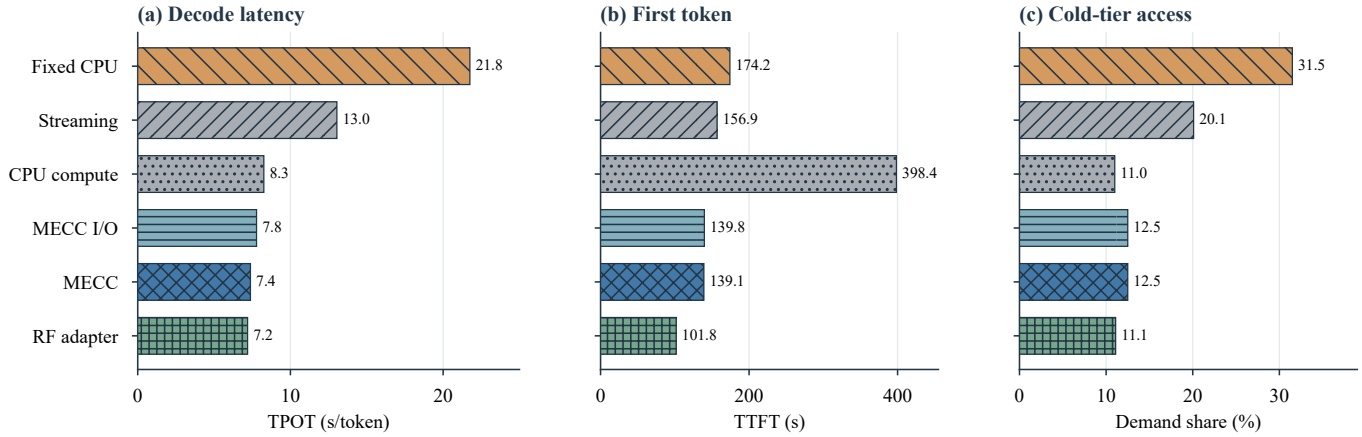


Fig. 4. Single-A100 Mixtral-8x7B comparison (eight WikiText-2 prompts, 64 output tokens, one run per policy). (a) Decode time per output token. (b) First-token latency. (c) Reported cold-tier demand share across all instrumented phases, including warmup and perplexity; it is not a decode-only share. All six local policies are shown.

B. Single-GPU Latency

Figure 4 reports TPOT of 7.38 s for MECC, 13.04 s for streaming, and 21.75 s for fixed-CPU retention. The corresponding TPOT ratios are 1.77× and 2.95×. The local RF adapter is slightly faster at 7.19 s; MECC I/O takes 7.78 s and the CPU-compute adapter takes 8.26 s. The cold-tier demand shares are 12.5% for MECC, 20.1% for streaming, 31.5% for fixed CPU, and 11.1% for RF. These counts span several phases, so they describe cache behavior rather than a phase-matched explanation of TPOT alone.

The first-token times are 139.1 s for MECC, 156.9 s for streaming, 174.2 s for fixed CPU, 101.8 s for RF, and 398.4 s for CPU compute. MECC, streaming, fixed CPU, RF, and MECC I/O report perplexity 3.427 on the same 1024-token sample; CPU compute reports 3.432. This quality comparison is specific to that sample. A separate short-window route/logit/token comparison supports the implementation check but is not an independent full-resident reference.

C. Retention Control

The independent-cache run matches the main runs' model, nominal slots, input scale, and uniform initialization. Figure 5 compares independent caching, fixed CPU, and joint MECC retention. Independent caching takes 32.94 s/token and reports 47.9% cold-tier demand; the fixed-CPU policy takes 21.75 s and reports 31.5%; MECC takes 7.38 s and reports 12.5%. This comparison supports the joint-retention mechanism under these local policies. Physical host footprints and exposed I/O time still need matched accounting before the entire latency gap can be assigned to replacement alone.

D. Host-Slot Sensitivity

The 30 and 60 GB nominal CPU settings use the same four-prompt, 16-token WikiText-2 protocol within each setting. They are a distinct experiment from the eight-prompt main comparison. At 30 GB, MECC takes 14.46 s/token and reports 32.2% cold-tier demand; at 60 GB, it takes 0.76 s/token and reports no cold-tier demand. The 60 GB plan has four HBM and four host slots per layer, enough for all eight experts. The RF adapter takes 13.06 s/token at 30 GB and 0.74 s/token at 60 GB, faster than MECC in both settings. Figure 6 shows all five local policies at these two settings.

E. Scope of the Measurements

These results are single-run observations on a RAID logical device reporting rotational media. The main comparison uses uniform initialization and does not measure the compiler's profile-guided gain. A true WikiText-2 to GSM8K task-switch sequence has now been implemented, but no completed task-switch result appears in the committed result set. The 20 GB budget runs mix prompt sources and cannot be joined to the 30/60 GB curve. Physical drive identity, physical host-cap equivalence, and multi-GPU resident performance remain separate measurements.

VII. Related Work

Sparse execution and serving. Switch and GShard decouple active computation from total parameters [2], [3]; DeepSeekMoE uses fine-grained and shared experts [5]. DeepSpeed-MoE optimizes distributed inference [16]; Tutel adapts parallelism/pipelining [17]; MegaBlocks targets sparse training kernels [18]. PagedAttention and SGLang optimize KV allocation and prefix reuse [19], [20], distinct from expert residency.

Heterogeneous memory. ZeRO-Infinity uses GPU/CPU/NVMe for training [21]; DeepSpeed Inference supports heterogeneous serving [7]. FlexGen plans tensor placement and throughput-oriented schedules [6]. LLM in a flash combines reuse windows with flash-friendly layouts [8]. PowerInfer splits hot/cold neuron computation across GPU/CPU [22].

Offloading. Fast MoE offloading and SwapMoE target memory-constrained serving [23], [24]. Fiddler uses popularity-based GPU placement and CPU expert computation [9]. MoE-Lightning searches batching, execution, and weight/KV placement under CPU/GPU constraints, excluding disk variables [10]. Klotski combines expert-aware multi-batch execution with constraint-sensitive planning and correlation-aware prefetching [25].

Expert prediction. Pre-gated MoE changes the gating design [26]; ProMoE predicts expert use from intermediate results [13]; MoE-Infinity uses activation traces for replacement/prefetch [12]. MECC's demand-only control excludes prediction; its overlap control uses IDs available from the unchanged router.

SSD-based serving. SSD-LLaMA combines expert-pack I/O, lossless transport, frequency/recency-based VRAM admission, and RAM LRU [11]. Transient experts may bypass retention. FlashMoE learns

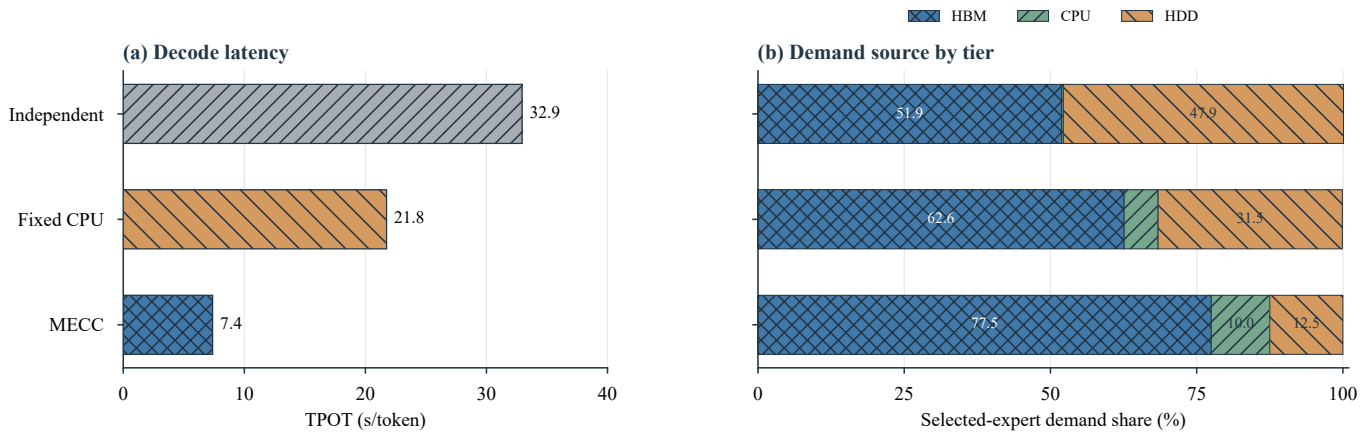


Fig. 5. Matched single-A100 cache-policy control at 50/40 GB nominal allowances, eight WikiText-2 prompts, and 64 output tokens. (a) Decode latency. (b) Selected-expert demand resolved from HBM, CPU, or HDD across all instrumented phases. Independent and fixed-CPU policies are local mechanisms. One run per policy.

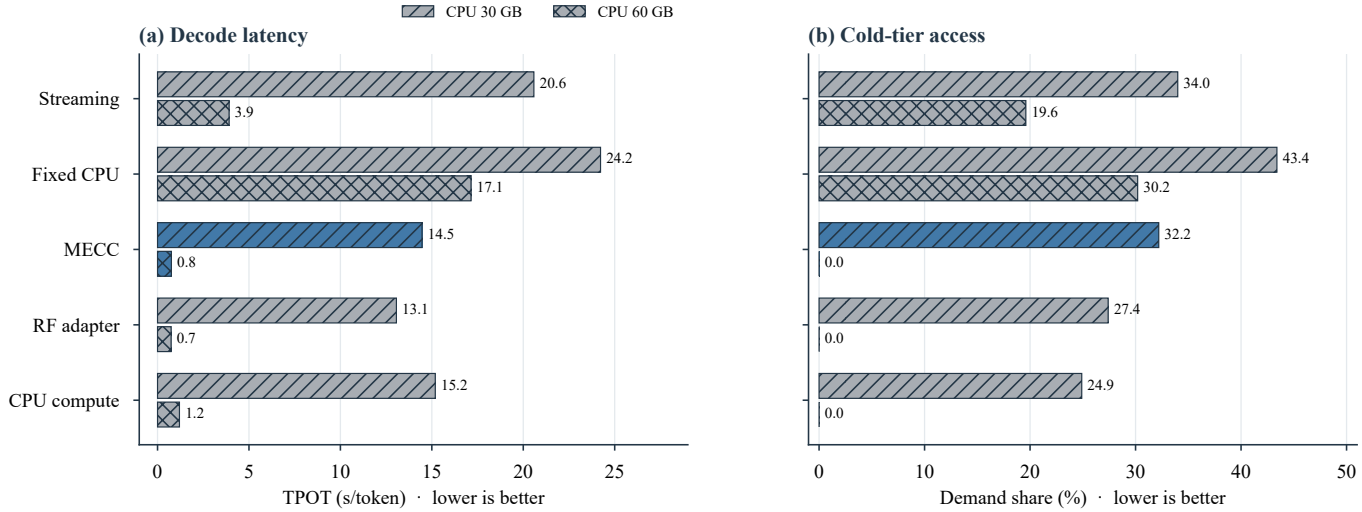


Fig. 6. Nominal CPU-slot sensitivity on one A100. Four WikiText-2 prompts and 16 output tokens per run; 30 and 60 GB are the only two settings shown with the same input source. The earlier 20 GB runs used a different prompt source and are excluded from this comparison. Cold-tier share covers all instrumented phases. One run per point.

a replacement predictor from recency/frequency signals and Belady-derived labels [14]. Plain RF/LRU omits these mechanisms.

Coordinated residency. MoE-CORE combines layer hit-rate curves, domain initialization, and routing history/scores [15]; optional substitution is separate from exact execution. ZipMoE combines lossless compression, compression-aware caching, and cache-affinity scheduling [27]. These overlaps require matched controls, not novelty claims based on module names.

The scope is HDD-backed single-GPU serving. Local adapters do not establish superiority over these systems.

VIII. Conclusion

On one A100 with uniform initial placement, MECC's joint-retention runtime reduces decode latency relative to local streaming, fixed-CPU, and independent-cache controls. The local RF adapter is slightly faster on the main workload. Profile-guided placement, task changes, physical budget equivalence, and multi-GPU tradeoffs require separate evidence before they enter the performance claim.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[4] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[5] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[6] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[7] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[8] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[9] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[10] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[11] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[12] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[13] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[14] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[15] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[16] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[17] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[18] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[19] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[20] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[21] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[22] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[23] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[24] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[25] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[26] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[27] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.